DOCUMENTACIÓN TÉCNICA Y EXPLICATIVA

SISTEMA INTEGRAL DEL INVENTARIO FORESTAL NACIONAL (IFN) – IDEAM

INTEGRANTES

DAVID SANTIAGO GUERRERO GARCIA 01250371065

NIKOL FERNANDA MEDINA MONTERREY 01250371052

PROFESOR

CARLOS ALBERTO CARRASCAL AVENDAÑO

UNIVERSIDAD DE SANTANDER

PROGRAMACION DE COMPUTADORES I

BUCARAMANGA SANTANDER

8 NOVIEMBRE 2026

SISTEMA INTEGRAL DEL INVENTARIO FORESTAL NACIONAL (IFN) – IDEAM

Proyecto: Sistema Integral del Inventario Forestal Nacional (IFN)

Lenguaje de programación: Python

Entorno de ejecución: Google Colab

Biblioteca de interfaz gráfica: IPyWidgets

1. INTRODUCCIÓN

El presente proyecto consiste en el desarrollo de un sistema informático para gestionar información relacionada con las inspecciones forestales del Inventario Forestal Nacional (IFN), tomando como referencia la gestión de información forestal del IDEAM en Colombia.

El sistema fue desarrollado utilizando el lenguaje de programación Python y la biblioteca IPyWidgets, que permite construir una interfaz gráfica interactiva dentro de un entorno Jupyter Notebook o JupyterLab.

Su propósito principal es facilitar el registro, almacenamiento, consulta y procesamiento de información obtenida durante las inspecciones forestales. Para ello, el programa permite ingresar datos de los conglomerados evaluados, registrar cantidades de árboles según su clasificación, identificar especies y sus usos, consultar inspecciones previamente almacenadas y generar informes con indicadores generales.

El desarrollo se organiza mediante una estructura de tres capas: datos, lógica de negocio y presentación. Esta organización permite separar las responsabilidades del programa y facilita su comprensión, mantenimiento y modificación.

2. OBJETIVO GENERAL

Desarrollar un sistema informático en Python que permita gestionar información de inspecciones forestales mediante una interfaz interactiva, facilitando el registro, almacenamiento, consulta y generación de indicadores generales del Inventario Forestal Nacional.

3. OBJETIVOS ESPECÍFICOS

- Implementar un formulario interactivo para registrar información de las inspecciones forestales.

- Almacenar los registros en un archivo de texto con persistencia de información.
- Desarrollar un mecanismo de búsqueda de inspecciones mediante el identificador del conglomerado.
- Validar los datos ingresados y los registros recuperados del archivo.
- Calcular indicadores generales relacionados con la cantidad de inspecciones, árboles, especies y usos registrados.
- Presentar los resultados mediante una interfaz gráfica organizada en diferentes pestañas.
- Aplicar una arquitectura de programación por capas para separar los procesos de almacenamiento, procesamiento y presentación.

4. TECNOLOGÍAS Y LIBRERÍAS UTILIZADAS

Para el desarrollo del sistema se utilizaron las siguientes herramientas:

4.1. Python

Es el lenguaje de programación utilizado para desarrollar toda la lógica del sistema. Permite crear funciones, procesar información, realizar operaciones matemáticas, manipular archivos y construir estructuras de datos.

4.2. IPyWidgets

Es una biblioteca que permite crear componentes interactivos dentro de Jupyter Notebook y JupyterLab.

En este proyecto se utiliza para construir los formularios, botones, campos de texto, listas desplegables, pestañas y espacios donde se presentan los resultados.

Entre los componentes utilizados se encuentran:

- widgets.Text: permite ingresar información mediante campos de texto.
- widgets.BoundedIntText: permite ingresar cantidades numéricas enteras dentro de un rango establecido.
- widgets.Dropdown: permite seleccionar una opción de una lista desplegable.
- widgets.Button: crea botones para ejecutar acciones.
- widgets.Output: permite mostrar mensajes y resultados.
- widgets.VBox: organiza elementos verticalmente.
- widgets.HBox: organiza elementos horizontalmente.
- widgets.Tab: permite organizar la interfaz en diferentes pestañas.
- widgets.HTML: permite presentar títulos, mensajes y tablas utilizando contenido HTML.

4.3. OS

Es una biblioteca estándar de Python utilizada para interactuar con el sistema operativo.

En el proyecto se utiliza para comprobar si el archivo de almacenamiento existe mediante la instrucción os.path.exists().

4.4. IPython.display

Se utiliza para controlar la presentación de elementos dentro del entorno Jupyter.

Se importan dos funciones:

- display(): permite mostrar componentes gráficos, mensajes y resultados.
- clear_output(): limpia el contenido anterior de un espacio de salida antes de presentar nuevos resultados.

4.5. Archivo TXT

Se utiliza como mecanismo de almacenamiento permanente de los registros.

El archivo se denomina inspecciones_ifn.txt y almacena la información de cada inspección en una línea, separando los nueve campos mediante el carácter |.

5. ARQUITECTURA DEL SISTEMA

El programa está organizado mediante una arquitectura de tres capas, en la que cada una cumple una responsabilidad específica.

5.1. Capa de datos

Es la encargada de realizar las operaciones de lectura y escritura de información en el archivo de texto.

Contiene las siguientes funciones:

guardar_registro_txt(registro_linea)

Recibe una cadena de texto con los datos de una inspección y la almacena al final del archivo inspecciones_ifn.txt.

Utiliza el modo de apertura "a", que permite agregar información sin eliminar los registros anteriores.

Si el almacenamiento se realiza correctamente, retorna True. Si ocurre un error de escritura, muestra un mensaje y retorna False.

leer_registros_txt()

Se encarga de recuperar la información almacenada en el archivo.

Primero verifica si el archivo existe. Si no existe, retorna una lista vacía. Si existe, abre el archivo en modo lectura y obtiene todas las líneas almacenadas.

Si ocurre un error durante la lectura, muestra un mensaje y retorna una lista vacía.

5.2. Capa de lógica de negocio

Es la encargada de procesar la información, organizar los registros, realizar búsquedas y calcular los indicadores.

Contiene las siguientes funciones:

convertir_linea_a_inspeccion(linea)

Convierte una línea de texto del archivo TXT en un diccionario de Python.

Primero elimina los espacios y saltos de línea innecesarios y posteriormente divide la información utilizando el carácter |.

El sistema espera encontrar exactamente nueve campos. Si la cantidad de campos es diferente, el registro se considera inválido.

También convierte los campos correspondientes a fustales, latizales y brinzales en valores enteros. Si alguno de estos datos no puede convertirse, retorna None.

procesar_nueva_inspeccion(...)

Recibe los nueve datos ingresados en el formulario de registro.

Organiza la información en una sola cadena de texto, separando cada campo mediante el carácter |.

Finalmente, envía el registro a la función guardar_registro_txt() para realizar su almacenamiento.

buscar_inspeccion_logica(idBuscado)

Permite buscar una inspección mediante el identificador del conglomerado.

Para realizar la búsqueda, obtiene los registros almacenados y los recorre uno por uno. Cada línea se convierte en un diccionario y posteriormente se compara su identificador con el ID solicitado.

Si encuentra una coincidencia, retorna el diccionario correspondiente. Si termina la búsqueda sin encontrar resultados, retorna None.

obtener_todas_las_inspecciones_logica()

Recupera todos los registros almacenados y convierte cada línea válida en un diccionario.

Los registros que presentan errores de estructura o conversión son descartados.

Finalmente, retorna una lista con todas las inspecciones válidas.

calcular_indicadores()

Es la función encargada de procesar la información general del inventario forestal.

Primero obtiene todas las inspecciones válidas. Si no existen registros, retorna None.

Cuando existen datos, realiza los siguientes cálculos:

- Total de inspecciones: cantidad de registros válidos.
- Total de árboles: suma de fustales, latizales y brinzales de todas las inspecciones.
- Densidad promedio: división del total de árboles entre el total de inspecciones.
- Especies presentes: conjunto de nombres de especies registradas, sin repetir.
- Usos registrados: conjunto de usos de las especies, sin repetir.

Finalmente, organiza las especies y los usos alfabéticamente y retorna los resultados en un diccionario.

5.3. Capa de presentación

Es la encargada de establecer la comunicación entre el usuario y el sistema mediante la interfaz gráfica desarrollada con IPyWidgets.

Esta capa contiene los formularios, campos de entrada, botones, pestañas y espacios de resultados.

Se encuentra organizada en tres funcionalidades principales:

- Registro de inspecciones.
- Consulta de inspecciones.
- Generación de informes e indicadores.

6. FUNCIONAMIENTO GENERAL DEL SISTEMA

Al ejecutar el programa, se importan las bibliotecas necesarias, se establece el nombre del archivo de almacenamiento y se configuran las funciones correspondientes a cada capa.

Posteriormente, se crean los componentes de la interfaz gráfica y se organizan en tres pestañas.

La función iniciar_interfaz() se encarga de mostrar la ventana principal del sistema, que contiene el encabezado institucional y las diferentes opciones de navegación.

El usuario puede seleccionar cualquiera de las tres funcionalidades según la operación que necesite realizar.

6.1. Módulo de registro de inspecciones

Este módulo permite ingresar y almacenar la información correspondiente a una nueva inspección forestal.

El formulario solicita los siguientes datos:

Campo	**Descripción**
ID del conglomerado	Identificador único utilizado para reconocer el conglomerado.
Geolocalización	Ubicación de las subparcelas evaluadas.
Nombre de la brigada	Nombre de la brigada asignada a la inspección.
Fustales	Cantidad de árboles clasificados como fustales.
Latizales	Cantidad de árboles clasificados como latizales.
Brinzales	Cantidad de árboles clasificados como brinzales.
Nombre común	Nombre común de la especie identificada.
Uso de la especie	Clasificación del uso de la especie.
Código de muestra	Código utilizado para identificar la muestra.

El campo correspondiente al uso de la especie utiliza una lista desplegable que contiene las siguientes opciones:

- Medicinal.
- Alimenticio.
- Maderable.
- No maderable.

Proceso de registro

El procedimiento comienza cuando el usuario completa los campos y presiona el botón "Registrar inspección".

La función registrar_inspeccion() recibe el evento del botón y limpia el espacio de salida para mostrar el resultado de la operación.

Posteriormente, comprueba que los campos de texto obligatorios no estén vacíos y que se haya seleccionado un uso de la especie.

Si falta algún dato, el sistema muestra un mensaje solicitando completar la información y detiene el proceso.

Si todos los campos están completos, se ejecuta procesar_nueva_inspeccion(), que organiza los datos y solicita su almacenamiento en el archivo TXT.

Cuando el almacenamiento es exitoso, se muestra un mensaje de confirmación y se limpian los campos del formulario para permitir registrar una nueva inspección.

Si ocurre un error durante el almacenamiento, se presenta un mensaje indicando que no fue posible guardar la información.

6.2. Módulo de consulta de inspecciones

Este módulo permite recuperar y visualizar una inspección previamente almacenada utilizando el ID del conglomerado.

La interfaz contiene un campo de texto para ingresar el identificador y un botón denominado "Consultar inspección".

Proceso de consulta

Cuando el usuario presiona el botón, se ejecuta la función consultar_inspeccion().

Inicialmente, se limpia el espacio de resultados y se obtiene el identificador ingresado.

Si el campo está vacío, el sistema muestra un mensaje solicitando ingresar el ID.

Si existe un identificador, se ejecuta la función buscar_inspeccion_logica() para realizar la búsqueda dentro del archivo de almacenamiento.

Durante el proceso, cada registro es leído y convertido en un diccionario. Los registros válidos se comparan con el identificador solicitado.

Si se encuentra una coincidencia, el sistema organiza los datos mediante un conjunto de etiquetas descriptivas y construye una tabla HTML para mostrar la información de la inspección.

Si no se encuentra ningún registro con el identificador ingresado, se presenta el mensaje: "No se encontró una inspección con ese ID".

6.3. Módulo de informes e indicadores

Este módulo permite obtener un resumen general de la información almacenada en el sistema.

La interfaz dispone de un botón denominado "Generar informe", que ejecuta la función generar_informe().

Proceso de generación de informes

Al presionar el botón, el sistema ejecuta la función calcular_indicadores() para recuperar y procesar todas las inspecciones válidas.

Si no existen inspecciones registradas, se muestra un mensaje indicando que no hay información disponible para calcular los indicadores.

Si existen registros, se inicializan las variables necesarias para realizar los cálculos.

El programa recorre cada inspección y realiza las siguientes operaciones:

1. Suma las cantidades de fustales, latizales y brinzales para obtener el total de árboles.
2. Identifica las especies presentes y las agrega a un conjunto para evitar duplicados.
3. Identifica los usos registrados y los agrega a otro conjunto para evitar repeticiones.
4. Cuenta el número total de inspecciones válidas.
5. Calcula la densidad promedio de árboles por conglomerado.

Fórmula utilizada para calcular la densidad promedio:

Densidad promedio = Total de árboles / Total de inspecciones

Por ejemplo, si existen 5 inspecciones y se registran 500 árboles en total:

Densidad promedio = 500 / 5 = 100 árboles por conglomerado.

Finalmente, el sistema presenta un informe regional que contiene:

- Total de conglomerados evaluados.
- Total de árboles contados.
- Densidad promedio de árboles por conglomerado.
- Especies presentes.
- Usos registrados.

7. ESTRUCTURA DEL ARCHIVO DE ALMACENAMIENTO

El sistema utiliza un archivo de texto denominado inspecciones_ifn.txt para conservar la información registrada.

Cada inspección se almacena en una línea independiente y contiene nueve campos separados por el símbolo |.

Ejemplo de un registro:

CONG-001|6.1234,-73.1234|Brigada Norte|15|20|10|Cedro|maderable|MUE-001

La estructura corresponde al siguiente orden:

1. ID del conglomerado.
2. Geolocalización.
3. Nombre de la brigada.
4. Cantidad de fustales.
5. Cantidad de latizales.
6. Cantidad de brinzales.
7. Nombre común de la especie.
8. Uso de la especie.
9. Código de muestra.

Este mecanismo permite conservar la información entre diferentes ejecuciones del programa, ya que los registros permanecen almacenados en el archivo de texto.

8. VALIDACIONES Y MANEJO DE ERRORES

El sistema incorpora diferentes mecanismos de validación para evitar que información incompleta o incorrecta afecte los procesos principales.

Las validaciones implementadas son las siguientes:

Validación del formulario de registro: comprueba que los campos de texto obligatorios no estén vacíos y que se haya seleccionado una opción para el uso de la especie.

Validación de cantidades numéricas: los campos de fustales, latizales y brinzales utilizan BoundedIntText con un valor mínimo de cero, evitando el ingreso de cantidades negativas mediante estos controles.

Validación de estructura de registros: verifica que cada línea recuperada del archivo contenga exactamente nueve campos.

Validación de conversión numérica: comprueba que las cantidades de fustales, latizales y brinzales puedan convertirse a números enteros.

Manejo de errores de escritura: utiliza try y except IOError para controlar posibles errores durante el almacenamiento.

Manejo de errores de lectura: controla posibles errores al abrir o leer el archivo de almacenamiento.

Validación de consultas: verifica que el usuario haya ingresado un identificador antes de iniciar la búsqueda.

Validación de información para informes: comprueba que existan inspecciones válidas antes de realizar los cálculos.

9. ORGANIZACIÓN DE LA INTERFAZ GRÁFICA

La interfaz principal se construye mediante la función iniciar_interfaz() y está compuesta por un encabezado y un sistema de pestañas.

El encabezado presenta el nombre del sistema y su relación con la gestión de inspecciones forestales del IDEAM Colombia.

Las pestañas se crean utilizando widgets.Tab() y se configuran mediante el método set_title().

La distribución es la siguiente:

Pestaña 1: Registrar

Contiene el formulario para ingresar los datos de una nueva inspección y el botón para almacenarla.

Pestaña 2: Consultar

Contiene el campo para ingresar el ID del conglomerado, el botón de búsqueda y el espacio donde se muestran los resultados.

Pestaña 3: Informes

Contiene el botón para generar el informe y el espacio destinado a presentar los indicadores forestales.

Esta organización facilita la navegación y permite que el usuario acceda directamente a la operación que necesita realizar.

10. VENTAJAS DEL SISTEMA

El desarrollo presenta diferentes ventajas relacionadas con la organización y gestión de la información forestal:

- **Organización de la información:** permite registrar los datos de las inspecciones mediante una estructura definida.
- **Persistencia de datos:** conserva los registros en un archivo TXT, incluso después de cerrar o reiniciar el programa.
- **Facilidad de consulta:** permite recuperar información mediante el ID del conglomerado.

- **Procesamiento automático:** calcula los indicadores generales sin necesidad de realizar las operaciones manualmente.
- **Interfaz interactiva:** facilita la utilización del sistema mediante formularios, botones y pestañas.
- **Separación de responsabilidades:** la arquitectura de tres capas facilita la comprensión y el mantenimiento del código.
- **Control de errores:** incorpora validaciones para evitar el procesamiento de registros incompletos o con una estructura incorrecta.

11. LIMITACIONES Y POSIBLES MEJORAS

Aunque el sistema cumple con las funcionalidades principales propuestas, existen aspectos que podrían mejorarse en futuras versiones.

Entre las posibles mejoras se encuentran:

- Implementar una base de datos para administrar una mayor cantidad de registros.
- Incorporar la edición y eliminación de inspecciones previamente almacenadas.
- Implementar una validación para evitar registros duplicados.
- Agregar filtros de búsqueda por especie, brigada o uso.
- Incorporar gráficos estadísticos para representar los resultados de los informes.
- Mejorar las validaciones de los campos numéricos y de geolocalización.
- Implementar mecanismos de seguridad y control de acceso para proteger la información.
- Permitir exportar los informes a formatos como PDF o Excel.

Estas mejoras permitirían ampliar las capacidades del sistema y facilitar su utilización en escenarios con mayores necesidades de gestión de información.

12. CONCLUSIONES

El desarrollo del Sistema Integral del Inventario Forestal Nacional permitió aplicar diferentes conceptos de programación utilizando Python, integrando el manejo de archivos, funciones, estructuras de datos, validaciones, operaciones matemáticas y construcción de interfaces gráficas.

La implementación de IPyWidgets permitió desarrollar una interfaz interactiva organizada en tres módulos principales: registro, consulta e informes. Esto

facilita la interacción del usuario y permite ejecutar las operaciones sin necesidad de modificar directamente el código.

La utilización de un archivo TXT como mecanismo de almacenamiento permitió implementar la persistencia de información, mientras que la organización mediante tres capas facilitó la separación de las operaciones de almacenamiento, procesamiento y presentación.

Finalmente, el sistema demuestra cómo los conocimientos de programación pueden aplicarse al desarrollo de herramientas informáticas orientadas a la organización y análisis de información forestal, proporcionando una solución funcional para gestionar registros de inspecciones y obtener indicadores generales del inventario.